

PROGRAMAÇÃO DE COMPUTADORES



Dicionários em Python

Prof. Dr. José Jairo de Santana e Silva

Aula sobre Estrutura Chave-Valor

Dicionários em Python: a estrutura chave-valor

AGENDA DA AULA

☰ Agenda

Dicionários em Python

- Por que precisamos de uma estrutura chave-valor
- Criação e acesso
- Adicionar, modificar e remover pares
- Métodos: `.keys`, `.values`, `.items`, `.get`, `.update`, `.pop`
- Iteração sobre dicionários
- Padrões comuns: contagem e agrupamento
- Dicionários aninhados
- Comparação: lista, tupla, dicionário
- Exercícios guiados e desafios

OBJETIVOS DE APRENDIZAGEM

🎯 Objetivos

Ao final desta aula, o estudante será capaz de:

- Criar **dicionários** e acessar valores por **chave**
- Adicionar, alterar e remover pares chave-valor
- Iterar sobre um dicionário com **for** e o método **.items()**
- Aplicar o padrão de **contagem** com dicionários
- Modelar registros com **dicionários aninhados**
- Comparar dicionário com lista e tupla, escolhendo a estrutura certa para cada problema
- Combinar dicionários com condicionais, laços e funções

ONDE VOCÊ VAI USAR ISSO?

Onde aparecem na prática

Dicionário é uma das estruturas mais usadas em Python para representar dados nomeados:

- **APIs REST e JSON:** respostas de APIs (Spotify, GitHub, Mercado Pago) chegam como dicionário aninhado.
- **Bancos NoSQL** (MongoDB, Firebase): cada documento é um dicionário.
- **Configurações:** `settings.py` do Django, variáveis de ambiente, arquivos `.json/.toml`.
- **Aplicações web:** sessões, cookies, cabeçalhos HTTP, contexto de templates.
- **Ciência de dados:** `pandas.DataFrame` se comporta como dicionário de colunas.
- **Estado de aplicação:** carrinho de compras, perfil de usuário, cache, ranking.

■ Em projetos web e de dados em Python, boa parte das estruturas em memória são dicionários ou listas de dicionários.

</> Exemplo real: resposta de uma API

Quando você chama uma API (ex.: GitHub), a resposta vem em **JSON** e é lida em Python como um **dicionário aninhado**.

Código:

```
resposta = {  
    "login": "octocat",  
    "nome": "The Octocat",  
    "publicos": {"seguidores": 9001},  
    "linguagens": ["Python", "Ruby"]  
}  
  
print(resposta["nome"])  
print(resposta["publicos"]["seguidores"])  
print(resposta["linguagens"][0])
```

Saída:

The Octocat

9001

Python

Esse é o padrão usado para consumir APIs REST em geral.

Ponte para o futuro: dicionário e classes

Em **POO**, classes e objetos têm sintaxe parecida com dicionário:

```
# Dicionário (hoje):
aluno = {"nome": "Ana", "nota": 8.5}
print(aluno["nome"])

# Classe e objeto (próximo passo):
class Aluno:
    def __init__(self, nome, nota):
        self.nome = nome
        self.nota = nota

ana = Aluno("Ana", 8.5)
print(ana.nome)
```

- Um objeto pode ser pensado como um **dicionário com comportamento** (métodos).
- O acesso é parecido: **dic["nome"]** vira **objeto.nome**.
- Frameworks como Django, Flask e FastAPI usam classes intensivamente, e internamente lidam com dicionários para configuração, requisições e respostas.

Conhecer bem dicionário facilita a transição para POO mais à frente.

POR QUE DICIONÁRIOS?

❓ Limite das listas

Imagine que queremos guardar a idade de várias pessoas:

```
nomes = ["Ana", "Bruno", "Carla"]  
idades = [21, 35, 19]
```

Para descobrir a idade da Carla, precisamos:

- localizar a **posição** da Carla em **nomes**
- usar essa posição em **idades**

O índice numérico só faz sentido quando a **posição** importa. Quando o que importa é o **nome** (a etiqueta do dado), trabalhar com índices é trabalhoso e propenso a erros.

💡 A ideia: indexar por nome

Um **dicionário** indexa os valores por **chave**, não por posição.

```
idades = {"Ana": 21, "Bruno": 35, "Carla": 19}  
print(idades["Carla"])    # 19
```

A chave é como uma **etiqueta** que aponta direto para o valor.

▮ Sempre que pensar "para cada X, eu quero saber Y", o dicionário é a estrutura natural.

 Analogia: dicionário de papel

Dicionário comum	Dicionário em Python
Palavra	Chave
Definição	Valor
Procurar pela palavra	Acessar pelo nome da chave
Cada palavra aparece uma vez	Cada chave é única

Em Python, escrevemos `dicionario[chave]` para "abrir" o dicionário naquela entrada.

CRIAÇÃO E ACESSO

✚ Criando dicionários

Dicionários são criados com **chaves** { } no formato **chave: valor**.

Código:

```
aluno = {  
    "nome": "Ana",  
    "matricula": "2026001",  
    "nota": 8.5  
}  
  
idades = {"Ana": 21, "Bruno": 35, "Carla": 19}  
  
vazio = {}  
  
print(aluno)  
print(idades)  
print(vazio)
```

Saída:

```
## {'nome': 'Ana', 'matricula': '2026001', 'nota': 8.5}  
## {'Ana': 21, 'Bruno': 35, 'Carla': 19}  
## {}
```

Cada par **chave: valor** é separado por vírgula. As chaves devem ser únicas.

Acessando valores

Acesse o valor pela chave entre colchetes:

Código:

```
aluno = {"nome": "Ana", "matricula": "2026001", "nota": 8.5}

print(aluno["nome"])
print(aluno["nota"])
print("Aluno:", aluno["nome"], "Nota:", aluno["nota"])
```

Saída:

```
## Ana
```

```
## 8.5
```

```
## Aluno: Ana Nota: 8.5
```

A sintaxe é parecida com a de listas, mas dentro dos colchetes vai a **chave**, não a posição.

⚠ Chave inexistente: **KeyError**

Tentar acessar uma chave que não existe gera erro:

```
aluno = {"nome": "Ana", "nota": 8.5}
print(aluno["idade"]) # KeyError: 'idade'
```

Solução segura: usar o método `.get()`, que retorna **None** (ou um valor padrão) se a chave não existir.

Código:

```
aluno = {"nome": "Ana", "nota": 8.5}

print(aluno.get("idade"))
print(aluno.get("idade", "Idade nao informada"))
print(aluno.get("nome", "Sem nome"))
```

Saída:

None

Idade nao informada

Ana

Use `.get()` quando a chave **pode** não existir e você não quer interromper o programa.

Modificando e adicionando pares

A atribuição `dic[chave] = valor` funciona em duas situações:

- se a chave **existe**, o valor é substituído
- se a chave **não existe**, um novo par é adicionado

Código:

```
aluno = {"nome": "Ana", "nota": 8.5}

aluno["nota"] = 9.0
aluno["idade"] = 21

print(aluno)
```

Saída:

```
## {'nome': 'Ana', 'nota': 9.0, 'idade': 21}
```

■ Dicionários são **mutáveis**: podemos mudar, acrescentar e remover pares depois de criados.

Removendo pares

Duas formas comuns:

Código:

```
estoque = {"caderno": 30, "lapis": 100, "borracha": 50}

del estoque["lapis"]
print(estoque)

removido = estoque.pop("borracha")
print("Removido:", removido)
print(estoque)
```

Saída:

```
## {'caderno': 30, 'borracha': 50}
```

```
## Removido: 50
```

```
## {'caderno': 30}
```

 `del` apenas remove. `.pop()` remove e devolve o valor.

🔍 Verificando se a chave existe

O operador `in` testa se uma chave está no dicionário.

Código:

```
aluno = {"nome": "Ana", "nota": 8.5}

print("nome" in aluno)
print("idade" in aluno)

if "nota" in aluno:
    print("Tem nota:", aluno["nota"])
else:
    print("Sem nota")
```

Saída:

```
## True
```

```
## False
```

```
## Tem nota: 8.5
```

Esse teste evita o `KeyError` antes mesmo da consulta.

 Pergunta rápida

Considere:

```
agenda = {"Ana": "9999-1111", "Bruno": "9999-2222"}  
  
agenda["Carla"] = "9999-3333"  
agenda["Ana"] = "8888-1111"  
del agenda["Bruno"]  
  
print(agenda)  
print(len(agenda))
```

Qual é a saída?

▮ Tente responder **antes** de rodar o código.

 Pratique: criação e acesso

- 1) Crie um dicionário **livro** com as chaves "**titulo**", "**autor**" e "**ano**". Imprima apenas o autor.
- 2) A partir do dicionário criado, adicione a chave "**paginas**" com valor **320**. Em seguida, imprima o dicionário completo.
- 3) Usando **.get()**, tente acessar a chave "**editora**" (que não existe), retornando "**Desconhecida**" como padrão.

✓ Respostas: criação e acesso

```
# 1) Criar o dicionário e acessar o autor
livro = {"titulo": "Dom Casmurro", "autor": "Machado de Assis", "ano": 1899}
print(livro["autor"])

# 2) Adicionar nova chave e imprimir
livro["paginas"] = 320
print(livro)

# 3) Acesso seguro com .get() e valor padrao
print(livro.get("editora", "Desconhecida"))
```

Machado de Assis

{'titulo': 'Dom Casmurro', 'autor': 'Machado de Assis', 'ano': 1899, 'paginas': 320}

Desconhecida

Note como **[chave]** daria **KeyError** em **editora**, enquanto **.get()** devolve o padrão sem interromper o programa.

MÉTODOS DE DICIONÁRIO

🔑 .keys(), .values() e .items()

Três visões do mesmo dicionário:

```
notas = {"Ana": 8.5, "Bruno": 6.0, "Carla": 9.2}  
print(notas.keys())  
print(notas.values())  
print(notas.items())
```

```
## dict_keys(['Ana', 'Bruno', 'Carla'])
```

```
## dict_values([8.5, 6.0, 9.2])
```

```
## dict_items([('Ana', 8.5), ('Bruno', 6.0), ('Carla', 9.2)])
```

▮ **.items()** devolve pares (**chave**, **valor**) e é a forma mais usada quando precisamos dos dois.

.update(): juntar dois dicionários

Adiciona ou sobrescreve pares a partir de outro dicionário.

Código:

```
aluno = {"nome": "Ana", "nota": 8.5}
extras = {"idade": 21, "nota": 9.0}

aluno.update(extras)
print(aluno)
```

Saída:

```
## {'nome': 'Ana', 'nota': 9.0, 'idade': 21}
```

▮ Onde a chave já existia, o valor é **substituído**. Onde não existia, é **acrescentado**.

☰ Resumo dos principais métodos

Operação	Sintaxe	O que faz
Acessar	<code>dic[chave]</code>	Devolve o valor (ou <code>KeyError</code>)
Acessar com padrão	<code>dic.get(chave, padrao)</code>	Devolve <code>padrao</code> se não existir
Adicionar/alterar	<code>dic[chave] = valor</code>	Insere ou sobrescreve
Remover	<code>del dic[chave]</code>	Apaga o par
Remover e obter	<code>dic.pop(chave)</code>	Apaga e devolve o valor
Existência	<code>chave in dic</code>	<code>True</code> ou <code>False</code>
Listar chaves	<code>dic.keys()</code>	Visão das chaves
Listar valores	<code>dic.values()</code>	Visão dos valores
Listar pares	<code>dic.items()</code>	Visão de <code>(chave, valor)</code>
Juntar	<code>dic.update(outro)</code>	Mescla dois dicionários
Tamanho	<code>len(dic)</code>	Número de pares

 Pratique: métodos

- 1) Dado `precos = {"caderno": 12.5, "lapis": 2.0}`, use `.get()` para obter o preço de `"borracha"` com padrão `0`.
- 2) Use `.update()` para acrescentar `{"borracha": 3.5, "lapis": 2.5}` ao dicionário `precos`. O que acontece com o preço do lápis?
- 3) Remova `"caderno"` de `precos` usando `.pop()` e guarde o valor removido em uma variável.

✓ Respostas: métodos

```
precos = {"caderno": 12.5, "lapis": 2.0}

# 1) .get() com padrao 0 quando a chave nao existe
print(precos.get("borracha", 0))

# 2) .update() insere "borracha" e sobrescreve "lapis"
precos.update({"borracha": 3.5, "lapis": 2.5})
print(precos)

# 3) .pop() remove e devolve o valor
valor = precos.pop("caderno")
print("Removido:", valor)
print(precos)
```

```
## 0
```

```
## {'caderno': 12.5, 'lapis': 2.5, 'borracha': 3.5}
```

```
## Removido: 12.5
```

```
## {'lapis': 2.5, 'borracha': 3.5}
```

No item 2, o preço do lápis foi **sobrescrito** de 2.0 para 2.5: `.update()` substitui chaves que já existem.

ITERAÇÃO

for em um dicionário

Por padrão, o **for** percorre as **chaves**:

Código:

```
notas = {"Ana": 8.5, "Bruno": 6.0, "Carla": 9.2}

for chave in notas:
    print(chave, "->", notas[chave])
```

Saída:

```
## Ana -> 8.5
## Bruno -> 6.0
## Carla -> 9.2
```

Funciona, mas dá mais trabalho: precisamos consultar o valor a cada passo.

for com .items()

Forma idiomática: pegar chave e valor de uma vez.

Código:

```
notas = {"Ana": 8.5, "Bruno": 6.0, "Carla": 9.2}

for nome, nota in notas.items():
    if nota >= 7:
        situacao = "aprovado"
    else:
        situacao = "recuperacao"
    print(f"{nome}: {nota} ({situacao})")
```

Saída:

```
## Ana: 8.5 (aprovado)
## Bruno: 6.0 (recuperacao)
## Carla: 9.2 (aprovado)
```

Padrão: `for chave, valor in dic.items():`. Lembra o **desempacotamento** de tuplas da aula passada.

Somando e contando valores

Código:

```
vendas = {"Ana": 1200, "Bruno": 800, "Carla": 1500, "Diego": 950}

total = 0
maior_vendedor = ""
maior_valor = 0
for nome, valor in vendas.items():
    total += valor
    if valor > maior_valor:
        maior_valor = valor
        maior_vendedor = nome

print("Total:", total)
print("Media:", total / len(vendas))
print("Maior vendedor:", maior_vendedor, "com", maior_valor)
```

Saída:

```
## Total: 4450
```

```
## Media: 1112.5
```

```
## Maior vendedor: Carla com 1500
```

 Pratique: iteração

Considere `idades = {"Ana": 21, "Bruno": 35, "Carla": 19}`.

- 1) Imprima cada pessoa no formato `Ana tem 21 anos`, usando `.items()`.
- 2) Calcule e imprima a idade média do grupo (some todas as idades e divida pela quantidade).
- 3) Imprima o nome da pessoa `mais nova` (menor idade).

✓ Respostas: iteração

```
idades = {"Ana": 21, "Bruno": 35, "Carla": 19}

# 1) Formato "Nome tem X anos"
for nome, idade in idades.items():
    print(f"{nome} tem {idade} anos")

# 2) Idade media
soma = 0
for idade in idades.values():
    soma += idade
print("Media:", soma / len(idades))

# 3) Pessoa mais nova
mais_nova = ""
menor_idade = 999
for nome, idade in idades.items():
    if idade < menor_idade:
        menor_idade = idade
        mais_nova = nome
print("Mais nova:", mais_nova, "com", menor_idade)
```

```
## Ana tem 21 anos
## Bruno tem 35 anos
## Carla tem 19 anos
```

```
## Media: 25.0
```

```
## Mais nova: Carla com 19
```

PADRÃO CLÁSSICO: CONTAGEM

☰ Contagem de ocorrências

Queremos contar **quantas vezes cada item aparece** em uma lista.

Ideia: para cada item, somar 1 no dicionário, criando a entrada se ainda não existir.

Código:

```
votos = ["A", "B", "A", "C", "A", "B", "C", "C", "A"]

contagem = {}
for voto in votos:
    if voto in contagem:
        contagem[voto] += 1
    else:
        contagem[voto] = 1

print(contagem)
```

Saída:

```
## {'A': 4, 'B': 2, 'C': 3}
```

Esse é um dos padrões mais úteis com dicionários: **contar coisas**.

⚡ Versão enxuta com `.get()`

`.get(chave, 0)` devolve `0` quando a chave ainda não existe, eliminando o `if`.

Código:

```
votos = ["A", "B", "A", "C", "A", "B", "C", "C", "A"]  
  
contagem = {}  
for voto in votos:  
    contagem[voto] = contagem.get(voto, 0) + 1  
  
print(contagem)
```

Saída:

```
## {'A': 4, 'B': 2, 'C': 3}
```

▮ Mesma lógica, mais limpa. Guarde esse padrão: ele vai aparecer várias vezes.

A Aplicação: contar letras

Código:

```
texto = "programacao"

contagem = {}
for letra in texto:
    contagem[letra] = contagem.get(letra, 0) + 1

for letra, qtd in contagem.items():
    print(letra, "->", qtd)
```

Saída:

```
## p -> 1
## r -> 2
## o -> 2
## g -> 1
## a -> 3
## m -> 1
## c -> 1
```

■ Mesma estrutura da contagem de votos, agora aplicada a uma string.

 Pratique: contagem

- 1) Dada `lista = [1, 2, 1, 3, 2, 1, 4]`, conte quantas vezes cada número aparece e mostre o dicionário resultante.
- 2) Dada a frase `"ada lovelace"`, conte quantas vezes a letra `"a"` aparece. Use o padrão `dic.get(chave, 0) + 1`.
- 3) Dada `notas = [7, 8, 6, 7, 9, 8, 7, 6, 7]`, descubra qual nota mais se repete.

✓ Respostas: contagem

```
# 1) Contagem por valor
lista = [1, 2, 1, 3, 2, 1, 4]
contagem = {}
for n in lista:
    contagem[n] = contagem.get(n, 0) + 1
print(contagem)

# 2) Quantas vezes "a" aparece em "ada lovelace"
frase = "ada lovelace"
contagem = {}
for letra in frase:
    contagem[letra] = contagem.get(letra, 0) + 1
print("a aparece", contagem.get("a", 0), "vezes")

# 3) Nota que mais se repete
notas = [7, 8, 6, 7, 9, 8, 7, 6, 7]
contagem = {}
for nota in notas:
    contagem[nota] = contagem.get(nota, 0) + 1
mais_comum = max(contagem, key=contagem.get)
print("Mais frequente:", mais_comum, "( ", contagem[mais_comum], "vezes )")
```

```
## {1: 3, 2: 2, 3: 1, 4: 1}
```

```
## a aparece 3 vezes
```

```
## Mais frequente: 7 ( 4 vezes )
```

No item 3, `max(dic, key=dic.get)` devolve a **chave** cujo valor (a contagem) é o maior.

DICIONÁRIOS ANINHADOS

Valor pode ser qualquer coisa

O valor de uma chave pode ser número, string, **lista**, **tupla** ou **outro dicionário**.

Código:

```
aluno = {  
    "nome": "Ana",  
    "notas": [8.5, 9.0, 7.5],  
    "endereco": {"rua": "Rua A", "numero": 100}  
}  
  
print(aluno["notas"])  
print(aluno["notas"][0])  
print(aluno["endereco"]["rua"])
```

Saída:

```
## [8.5, 9.0, 7.5]
```

```
## 8.5
```

```
## Rua A
```

Encadear colchetes: `aluno["endereco"]["rua"]` lê "abra o endereço, depois a rua".

 Cadastro: lista de dicionários

A combinação mais frequente: uma **lista** em que cada elemento é um **dicionário** (um registro).

Código:

```
alunos = [  
    {"nome": "Ana", "nota": 8.5},  
    {"nome": "Bruno", "nota": 6.0},  
    {"nome": "Carla", "nota": 9.2}  
]  
  
for aluno in alunos:  
    if aluno["nota"] >= 7:  
        print(aluno["nome"], "aprovado")  
    else:  
        print(aluno["nome"], "recuperacao")
```

Saída:

```
## Ana aprovado  
## Bruno recuperacao  
## Carla aprovado
```

Modelo recorrente em sistemas reais: cada registro é um dicionário; a coleção é uma lista.

Código junto: turma como lista de dicionários

Digite e execute:

```
turma = [  
    {"nome": "Ana", "nota": 8.5},  
    {"nome": "Bruno", "nota": 5.5},  
    {"nome": "Carla", "nota": 9.0},  
    {"nome": "Diego", "nota": 4.0}  
]  
  
soma = 0  
for aluno in turma:  
    soma += aluno["nota"]  
media = soma / len(turma)  
  
print("Media da turma:", round(media, 2))  
  
print("Alunos acima da media:")  
for aluno in turma:  
    if aluno["nota"] > media:  
        print("-", aluno["nome"], aluno["nota"])
```

Depois altere:

- Adicione mais alunos
- Mude o critério para `nota >= 7`
- Imprima o aluno com a maior nota

 Pratique: aninhados

- 1) Crie `cliente = {"nome": "Ana", "compras": [50, 80, 120]}`. Imprima o valor total das compras.
- 2) Dada `lista_alunos = [{"nome": "Ana", "nota": 7}, {"nome": "Bruno", "nota": 9}, {"nome": "Carla", "nota": 5}]`, imprima o nome do aluno com a maior nota.
- 3) No mesmo `lista_alunos`, imprima quantos alunos estão aprovados (nota maior ou igual a 7).

✓ Respostas: aninhados

```
# 1) Total das compras do cliente
cliente = {"nome": "Ana", "compras": [50, 80, 120]}
total = 0
for valor in cliente["compras"]:
    total += valor
print(cliente["nome"], "comprou R$", total)

# 2) Aluno com maior nota
lista_alunos = [{"nome": "Ana", "nota": 7},
                 {"nome": "Bruno", "nota": 9},
                 {"nome": "Carla", "nota": 5}]
melhor = lista_alunos[0]
for a in lista_alunos:
    if a["nota"] > melhor["nota"]:
        melhor = a
print("Melhor aluno:", melhor["nome"], "com", melhor["nota"])

# 3) Quantos aprovados
aprovados = 0
for a in lista_alunos:
    if a["nota"] >= 7:
        aprovados += 1
print("Aprovados:", aprovados)
```

Ana comprou R\$ 250

Melhor aluno: Bruno com 9

Aprovados: 2

COMPARANDO ESTRUTURAS

Lista, tupla e dicionário

Característica	Lista []	Tupla ()	Dicionário { }
Ordem	Posição (0, 1, 2...)	Posição (0, 1, 2...)	Chave nomeada
Pode mudar?	Sim	Não	Sim
Acesso	<code>lista[2]</code>	<code>tupla[2]</code>	<code>dic["chave"]</code>
Boa para	Coleções que crescem	Registros fixos curtos	Mapear nome para valor
Exemplo típico	Lista de notas	<code>(x, y)</code>	<code>{"nome": "Ana", "nota": 8.5}</code>

Regra prática:

Se o que importa é a **posição**, use lista. Se é um **registro fixo curto**, tupla. Se é um **mapeamento etiqueta para valor**, dicionário.

👍 Boas práticas

- Use **chaves descritivas** ("nota", "idade"), não números mágicos
- Prefira **.get()** quando a chave **pode** não existir
- Prefira **for chave, valor in dic.items():** em vez de indexar manualmente
- Para contar coisas, use **dic[x] = dic.get(x, 0) + 1**
- Documente o **formato** esperado: o que é chave, o que é valor
- Evite chaves muito longas ou redundantes ("nome_do_aluno" quando "nome" basta)

PAUSA PRÁTICA

 Pergunta rápida

Considere:

```
precos = {"caderno": 12.5, "lapis": 2.0, "borracha": 3.5}
carrinho = ["caderno", "lapis", "lapis", "borracha", "caderno"]

total = 0
for item in carrinho:
    total += precos[item]

print(round(total, 2))
```

Qual é a saída?

Resposta esperada: 32.5.

 Exercícios da seção

1. Crie um dicionário `aluno` com `nome`, `matricula` e `nota`. Imprima cada campo em uma linha.
2. Dado `idades = {"Ana": 21, "Bruno": 35, "Carla": 19}`, imprima cada pessoa e indique se é maior de idade.
3. Considere uma lista de palavras. Crie um dicionário com a quantidade de vezes que cada palavra aparece.
4. Dado um dicionário de produtos e quantidades, imprima apenas os produtos com quantidade menor que 10.

EXERCÍCIOS

☰ Lista de Exercícios

- 1) Crie um dicionário **pessoa** com **nome**, **idade** e **cidade**. Imprima um relatório no formato **Nome: ... | Idade: ... | Cidade:**
- 2) Dado **notas** = {"Ana": 8.5, "Bruno": 6.0, "Carla": 9.2, "Diego": 5.5}, calcule e imprima a média da turma.
- 3) A partir do mesmo dicionário **notas**, imprima apenas os aprovados (**nota** >= 7).
- 4) Crie um dicionário a partir de duas listas paralelas **produtos** e **precos**. Use **zip** se já conhecer; caso contrário, use índice.
- 5) Dada uma string, conte quantas vezes cada **vogal** aparece e armazene em um dicionário.
- 6) Dada uma lista de números, construa um dicionário com chaves **"par"** e **"impar"** indicando quantos números de cada tipo existem.
- 7) Receba uma lista de dicionários de alunos com **nome** e **nota**. Imprima o nome do aluno com a **maior nota**.
- 8) Crie um dicionário **estoque** com 5 produtos e quantidades. Permita o usuário digitar um produto e uma quantidade vendida; atualize o estoque.
- 9) Dado um dicionário **agenda** com **nome** -> **telefone**, escreva um programa que pergunte um nome ao usuário e imprima o telefone (ou **Nao cadastrado**).
- 10) Dado um dicionário **precos** e uma lista **carrinho** de itens, calcule o total da compra. Itens fora de **precos** devem ser ignorados, com aviso.

DESAFIOS EXTRAS

Desafios complementares

D1) Inverter dicionário: dado `idades = {"Ana": 21, "Bruno": 35, "Carla": 19}`, gere um novo dicionário com `idade -> nome`. O que acontece se duas pessoas tiverem a mesma idade?

D2) Agrupar por categoria: dada uma lista de tuplas (`produto, categoria`), construa um dicionário no formato `categoria -> lista de produtos`.

D3) Ranking: dado um dicionário de pontuações `{nome: pontos}`, imprima os 3 maiores pontuadores em ordem decrescente.

D4) Mesclar agendas: dadas duas agendas em formato de dicionário, una as duas. Quando houver chaves repetidas, mantenha o valor da segunda.

D5) Histograma: receba uma lista de notas inteiras (0 a 10) e imprima um dicionário com a quantidade de notas em cada valor, do menor para o maior.

→ Na próxima aula

Vamos começar **importação e exportação de arquivos em Python**: ler dados de **.txt** e **.csv**, salvar resultados em disco e construir programas que **lembram do que aconteceu** entre execuções.

- Como abrir um arquivo com **open()** e o bloco **with**
- Ler linha a linha um **.txt**
- Ler um **.csv** e transformar cada linha em um **dicionário**
- Salvar listas e dicionários em arquivo

Vou enviar antes da aula um arquivo **.csv** para vocês baixarem e usarmos como exemplo prático.

RESUMO

Resumo da Aula

- Dicionários `{ }` armazenam dados no formato **chave-valor**
- Chaves são únicas e funcionam como **etiquetas**, não como posições
- Acesso: `dic[chave]`; com padrão: `dic.get(chave, padrao)`
- São **mutáveis**: `dic[chave] = valor` insere ou substitui; `del/.pop` removem
- **for chave, valor in dic.items()**: é o padrão idiomático de iteração
- **Padrão de contagem**: `dic[x] = dic.get(x, 0) + 1`
- Dicionários **aninhados** e **listas de dicionários** modelam registros e cadastros
- Escolha da estrutura: **lista** para posição, **tupla** para registro fixo, **dicionário** para mapeamento por nome

📖 Referências Bibliográficas

- FORBELLONE, A. L. V.; EBERSPACHER, H. F. **Lógica de Programação**: A Construção de Algoritmos e Estruturas de Dados. 4. ed. São Paulo: Pearson, 2016.
- MENEZES, N. N. C. **Introdução a Programação com Python**: Algoritmos e Lógica de Programação para Iniciantes. 4. ed. Rio de Janeiro: Novatec, 2024.
- Documentação Python, seção **Mapping Types - dict**: docs.python.org/3/library/stdtypes.html#mapping-types-dict
- Documentação Python, tutorial sobre **Dictionaries**: docs.python.org/3/tutorial/datastructures.html#dictionaries

Obrigado!

Prof. Dr. José Jairo de Santana e Silva

Universidade Positivo - 2026/1